

Funciones Smalltalk.st

```
1 "Select:"
2 "Función: Filtra elementos de una colección según una condición.
3 Devuelve: Nueva colección con los elementos que cumplen la condición.
4 Ejemplo: Obtener números pares de una lista."
5 | numbers evenNumbers |
6 numbers := #(1 2 3 4 5 6 7 8 9 10).
7 evenNumbers := numbers select: [:each | each even].
8 "evenNumbers tendrá el valor #(2 4 6 8 10)"
9
10 "collect:"
11 "Función: Transforma cada elemento de una colección.
12 Devuelve: Nueva colección con los elementos transformados.
13 Ejemplo: Duplicar cada número en una lista."
14 | numbers doubledNumbers |
15 numbers := #(1 2 3 4 5).
16 doubledNumbers := numbers collect: [:each | each * 2].
17 "doubledNumbers tendrá el valor #(2 4 6 8 10)"
18
19 "reduce:"
20 "Función: Reduce una colección a un solo valor aplicando una operación sucesiva.
21 Devuelve: Un único valor (acumulado).
22 Ejemplo: Sumar todos los números de una lista."
23 | numbers sum |
24 numbers := #(1 2 3 4 5).
25 sum := numbers reduce: [:acc :each | acc + each].
26 "sum tendrá el valor 15 (1 + 2 + 3 + 4 + 5)"
27
28 "inject:into:"
29 "Función: Similar a reduce:, pero permite inicializar un valor acumulador antes de empezar.
30 Devuelve: Un único valor (acumulado) a partir del valor inicial.
31 Ejemplo: Calcular el producto de todos los números con un valor inicial."
32 | numbers product |
33 numbers := #(1 2 3 4 5).
34 product := numbers inject: 1 into: [:acc :each | acc * each].
35 "product tendrá el valor 120 (1 * 2 * 3 * 4 * 5)"
36
37 "reduceRight:"
38 "Función: Similar a reduce:, pero aplica la operación de derecha a izquierda.
39 Devuelve: Un único valor (acumulado), considerando el orden inverso.
40 Ejemplo: Restar números de derecha a izquierda (simulado en Smalltalk)."
41 | numbers difference |
42 numbers := #(1 2 3 4 5).
43 difference := (numbers reversed) reduce: [:acc :each | acc - each].
44 "difference tendrá el valor 3 (5 - 4 - 3 - 2 - 1)"
45
46 "Do:"
47 "Función: Ejecuta una acción sobre cada elemento de la colección (efecto secundario).
48 Devuelve: Nada (a menudo utilizado para efectos secundarios, como imprimir).
49 Ejemplo: Mostrar cada elemento en la consola."
50 | numbers |
51 numbers := #(1 2 3 4 5).
```

```
52 numbers do: [:each | Transcript show: each printString; cr].
53 "Mostrará cada número en la consola (Transcript)"
54
55 "detect:
56 Función: Encuentra el primer elemento en una colección que cumple con una condición.
57 Devuelve: El primer elemento que cumple la condición o un valor alternativo si no se
  encuentra ninguno.
58 Ejemplo: Encontrar el primer número impar en una lista.
59 8. keysDo:
60 Función: Itera sobre todas las claves (keys) de un diccionario (o colección asociativa).
61 Devuelve: Nada (iteración con efectos secundarios).
62 Ejemplo: Imprimir todas las claves en un diccionario.
63 9. keysAndValuesDo:
64 Función: Itera sobre pares clave-valor en un diccionario.
65 Devuelve: Nada (iteración con efectos secundarios).
66 Ejemplo: Mostrar tanto las claves como los valores en un diccionario."
67
68 "select: y collect: trabajan con colecciones y devuelven nuevas colecciones.
69 reduce: y inject:into: acumulan resultados en un solo valor.
70 do: itera sobre elementos pero no devuelve una nueva colección ni un valor.
71 reduceRight: es similar a reduce: pero en sentido inverso (derecha a izquierda)."
```